■ Table of Contents

1. Introduction to Operating Systems
2. Process Management
3. Threads & Concurrency
4. CPU Scheduling
5. Process Synchronisation
6. Deadlocks
7. Memory Management
8. Virtual Memory & Paging
9. File Systems
10. I/O Systems & Storage
11. Types of OS
12. Security & Protection
13. Interview Q&A; — 40 Questions
14. My Notes Space

1. Introduction to Operating Systems

An **Operating System (OS)** is system software that manages computer hardware and software resources and provides common services for computer programs. It acts as an intermediary between users and the computer hardware.

■ Goals of an OS

- Convenience — make the computer easy to use
- Efficiency — use hardware resources efficiently
- Ability to evolve — allow new features to be added without disrupting existing services

■ Key Components

Component	Role
Kernel	Core of OS; manages resources directly
Shell	Interface between user and kernel (CLI/GUI)
File System	Organises and stores data on disk
Device Drivers	Communicate with hardware peripherals
System Calls	API for user programs to request OS services

■ System Call Types

- Process control (fork, exec, exit, wait)
- File management (open, read, write, close)
- Device management (ioctl, read, write)
- Information maintenance (getpid, alarm, sleep)
- Communication (pipe, shmget, mmap)

✏■ My Notes — Introduction

Watch video first, then write key takeaways here:

YouTube: youtube.com/@shivanshvasu

_____

_____

_____

2. Process Management

A **process** is a program in execution. It is the basic unit of work in an OS. Unlike a program (passive), a process is active and requires resources: CPU, memory, files, I/O devices.

■ Process States (5-State Model)

State	Description
New	Process is being created
Ready	Waiting to be assigned to CPU
Running	Instructions are being executed
Waiting/Blocked	Waiting for I/O or event to complete
Terminated	Process has finished execution

■ PCB — Process Control Block

The OS maintains a **PCB** for each process. It contains:

- Process ID (PID)
- Process state
- Program counter
- CPU registers
- Memory management info
- I/O status info
- Accounting info (CPU time used, time limits)

■ Context Switching

When the CPU switches from one process to another, the OS must **save the state (context)** of the old process and **load the saved context** of the new process. This is called a context switch and involves overhead — no useful work is done during switching.

✏■ My Notes — Processes

Key concepts I learned:

Doubts to clarify:

_____

_____

_____

3. Threads & Concurrency

A **thread** is the smallest unit of CPU execution. A process can have multiple threads — they share the process's memory space (code, data, heap) but each has its own stack and registers.

■ Process vs Thread

Feature	Process	Thread
Memory	Separate address space	Shared address space
Creation	Heavy (expensive)	Light (cheap)
Communication	IPC (complex)	Direct (via shared memory)
Crash impact	Only that process	Can crash entire process
Context switch	Slow	Fast

■ User Threads vs Kernel Threads

- **User threads**: managed above kernel level (faster, but if one blocks, all block)
- **Kernel threads**: managed by OS (slower creation but true parallelism)
- **Multithreading models**: Many-to-One, One-to-One, Many-to-Many

4. CPU Scheduling

CPU Scheduling decides which process from the ready queue gets the CPU. The goal: maximise CPU utilisation and throughput while minimising waiting and response times.

■ Scheduling Algorithms

Algorithm	Type	Key Idea	Drawback
FCFS	Non-preemptive	First come, first served	Convoy effect
SJF	Non-preemptive	Shortest job first	Starvation
SRTF	Preemptive	Shortest remaining time	Overhead
Round Robin	Preemptive	Time quantum for each	High overhead if small TQ
Priority	Both	Higher priority runs first	Starvation
MLFQ	Preemptive	Multiple queues by priority	Complex

■ Key Metrics

- **Turnaround Time** = Completion – Arrival
- **Waiting Time** = Turnaround – Burst time
- **Response Time** = First response – Arrival
- **Throughput** = Processes completed per unit time

✏■ My Notes — CPU Scheduling

Practice Gantt charts here:

Formula summary:

_____

_____

_____

5. Process Synchronisation

When multiple processes share resources, we need synchronisation to avoid **race conditions** — where the outcome depends on the order of execution.

■ Critical Section Problem

Three requirements for a valid solution:

- **Mutual Exclusion**: Only one process in critical section at a time
- **Progress**: If no process is in CS, selection must not be postponed indefinitely
- **Bounded Waiting**: A limit on how many times others can enter before a waiting process

■ Synchronisation Mechanisms

- **Mutex (Mutual Exclusion Lock)**: Binary lock — acquire/release
- **Semaphore**: Integer variable; wait() decrements, signal() increments
- **Monitor**: High-level construct with condition variables
- **Spinlock**: Busy-wait until lock is free (good for short CS on multiprocessors)

■■ Classic Problems

- **Producer-Consumer**: Bounded buffer shared between producer and consumer
- **Readers-Writers**: Multiple readers allowed; only one writer at a time
- **Dining Philosophers**: 5 philosophers, 5 forks — illustrates deadlock and starvation

6. Deadlocks

A **deadlock** is a situation where a set of processes are blocked, each waiting for a resource held by another process in the set — creating a circular wait.

4 Necessary Conditions (Coffman Conditions)

Condition	Meaning
Mutual Exclusion	Resources are non-shareable
Hold and Wait	Process holds resource while waiting for another
No Preemption	Resources can't be forcibly taken
Circular Wait	P1 waits for P2, P2 waits for P1, ...

■■ Handling Deadlocks

- **Prevention**: Eliminate one of the 4 conditions (e.g., resource ordering for circular wait)
- **Avoidance**: Banker's Algorithm — allocate only if safe state is maintained
- **Detection & Recovery**: Allow deadlock, detect it, then recover (kill process / preempt resource)
- **Ignorance (Ostrich Algorithm)**: Ignore the problem (used in UNIX/Windows for simplicity)

✏■ My Notes — Deadlocks

Banker's Algorithm steps:

My understanding:

_____

_____

_____

7. Memory Management

Memory management is the OS function handling or managing primary memory. It involves tracking which parts of memory are in use, allocating/deallocating memory to processes, and ensuring processes don't interfere with each other.

■ Contiguous Allocation

- **Fixed Partitioning**: Memory divided into fixed equal parts — suffers from internal fragmentation
- **Dynamic Partitioning**: Partitions sized to process — external fragmentation
- **First Fit**: Allocate first hole that is big enough
- **Best Fit**: Allocate smallest hole that fits — leaves tiny unusable holes
- **Worst Fit**: Allocate largest hole — leaves largest remaining hole

■ Fragmentation

- **Internal Fragmentation**: Allocated memory is larger than requested (wasted inside block)
- **External Fragmentation**: Enough total free memory but not contiguous
- **Compaction**: Shuffle memory to combine free space (expensive)

■ Segmentation

Memory is divided into logical segments (code, stack, heap, data). Each segment has a base and limit. The **Segment Table** maps logical addresses to physical.

8. Virtual Memory & Paging

Virtual Memory allows a process to execute even if it is not completely in physical memory. The OS gives each process the illusion of having a large contiguous private address space.

■ Paging

- Physical memory divided into fixed-size blocks called **frames**
- Logical memory divided into same-size blocks called **pages**
- OS maintains a **page table** mapping pages to frames
- Eliminates external fragmentation; causes minor internal fragmentation

■ Page Replacement Algorithms

Algorithm	Policy	Advantage
FIFO	Replace oldest page	Simple
Optimal	Replace page not used for longest future	Best performance (theoretical)
LRU	Replace least recently used page	Near-optimal, practical
LFU	Replace least frequently used	Good for frequency patterns
Clock (Second Chance)	FIFO with reference bit	Efficient approximation of LRU

■■ Thrashing

Thrashing occurs when a process spends more time paging than executing. It happens when there are too many processes and not enough frames. Fix: reduce degree of multiprogramming or use working-set model.

✏■ My Notes — Memory & Virtual Memory

TLB & page table questions:

Page fault calculation practice:

_____

_____

_____

9. File Systems

A **File System** organises how data is stored and retrieved on a storage device. It manages files (named collections of data) and directories.

■ File Attributes & Operations

- Attributes: Name, identifier, type, location, size, protection, timestamps
- Operations: Create, Read, Write, Seek, Delete, Truncate, Append

■ Directory Structure

- **Single-Level**: All files in one directory — naming conflicts
- **Two-Level**: One directory per user
- **Tree-Structured**: Hierarchical (most common — Unix/Windows)
- **Acyclic Graph**: Shared subdirectories (hard/soft links)
- **General Graph**: Allows cycles — requires garbage collection

■ Disk Allocation Methods

- **Contiguous**: Fast sequential access; external fragmentation
- **Linked**: No fragmentation; no direct access, slow
- **Indexed**: Index block holds pointers; supports direct access

■■ Common File Systems

- **FAT32**: Older Windows; simple but max 4GB file size
- **NTFS**: Modern Windows; journaling, permissions, large files
- **ext4**: Linux standard; journaling, large volume support
- **APFS**: Apple; optimised for SSDs, strong encryption

10. I/O Systems & Storage

The OS manages I/O devices through device drivers and controllers. Goals: efficiency (keep devices busy) and generality (uniform device interface).

I/O Techniques

- **Programmed I/O (Polling)**: CPU continuously checks device status — wastes CPU
- **Interrupt-Driven I/O**: Device interrupts CPU when ready — more efficient
- **DMA (Direct Memory Access)**: Device controller transfers data directly to memory — CPU free

Disk Scheduling Algorithms

Algorithm	Description
FCFS	Service in arrival order — simple but slow
SSTF	Shortest Seek Time First — may cause starvation
SCAN (Elevator)	Head moves one direction servicing, then reverses
C-SCAN	Circular SCAN — uniform wait times
LOOK/C-LOOK	Like SCAN but only goes as far as last request

11. Types of Operating Systems

■ Batch OS

Jobs collected into batches and processed — no user interaction (e.g., early IBM)

■ Time-Sharing OS

Multiple users share CPU time; fast context switching (e.g., Unix)

■ Distributed OS

Resources of multiple computers networked together as one system

■ Real-Time OS (RTOS)

Strict timing constraints; used in embedded systems, avionics (e.g., FreeRTOS, VxWorks)

■ Network OS

Provides networking features; runs on servers (e.g., Windows Server, Novell NetWare)

■ Mobile OS

Optimised for mobile hardware (e.g., Android, iOS)

■ Embedded OS

Tiny footprint for embedded devices (e.g., Contiki, INTEGRITY)

■ Multiprocessor OS

Manages multiple CPUs — symmetric (SMP) or asymmetric

12. Security & Protection

■■ Protection Goals

- Prevent unauthorised access to resources
- Ensure processes only access resources they are permitted to use
- Detect and report security violations

■ Key Concepts

- **Access Control List (ACL)**: Per-resource list of who can do what
- **Capability List**: Per-process list of permitted operations
- **Authentication**: Who are you? (password, biometric, token)
- **Authorisation**: What are you allowed to do?
- **Encryption**: Protect data at rest and in transit
- **Firewall**: Filter network traffic based on rules

■ Common Threats

- Buffer Overflow
- Privilege Escalation
- Denial of Service (DoS)
- Malware (Virus, Worm, Trojan, Ransomware)
- SQL Injection
- Man-in-the-Middle

13. ■ Interview Q&A; — 40 Questions

These are the most commonly asked OS interview questions at top tech companies. Study each answer thoroughly and try to explain in your own words!

Q1.

What is an Operating System and what are its main functions?

Ans: An OS is system software that manages hardware and software resources. Main functions: Process management (scheduling, creation, termination), Memory management (allocation, deallocation), File system management, I/O device management, Security and access control, and providing a User Interface.

Q2.

What is the difference between a process and a program?

Ans: A **program** is a passive entity — a set of instructions stored on disk. A **process** is an active entity — a program in execution. A program becomes a process when loaded into memory. One program can create multiple processes (e.g., multiple Chrome windows).

Q3.

What is a PCB (Process Control Block)?

Ans: PCB is a data structure maintained by the OS for each process. It stores: Process ID, state, program counter, CPU registers, memory management info (page table base, segment table), I/O status (open file list), and accounting info. The OS uses PCB for context switching.

Q4.

What is a context switch?

Ans: Context switch is the mechanism of saving the state (context) of a running process into its PCB, and loading the context of the next scheduled process. It incurs overhead as the CPU does no useful work during switching. Frequency depends on scheduler and time quantum.

Q5.

What is the difference between preemptive and non-preemptive scheduling?

Ans: Preemptive: OS can forcibly remove a running process from CPU (e.g., Round Robin, SRTF). Suitable for time-sharing and real-time systems.

Ans: Non-preemptive: Once a process gets CPU, it runs until it voluntarily releases (e.g., FCFS, SJF). Simpler but can cause starvation.

Q6.

What is a deadlock? State the necessary conditions.

Ans: A deadlock is a state where processes are permanently blocked, each waiting for a resource held by another. The 4 necessary conditions (Coffman): **Mutual Exclusion** (non-shareable resource), **Hold and Wait** (holding while waiting), **No Preemption** (can't forcibly take), **Circular Wait** (circular chain of waiting).

Q7.

What is the Banker's Algorithm?

Ans: The Banker's Algorithm is a deadlock avoidance algorithm by Dijkstra. It simulates resource allocation and only grants a request if the resulting state is 'safe' — meaning all processes can eventually complete. It requires advance knowledge of maximum resource needs. Key data structures: Available, Max, Allocation, Need matrices.

Q8.

What is the difference between a mutex and a semaphore?

Ans: Mutex: Binary lock (locked/unlocked). Only the thread that locked it can unlock it. Used for mutual exclusion. Has ownership concept.

Ans: Semaphore: Integer variable (0 to N). Can be incremented by any thread. Used for signalling and resource counting. Counting semaphore allows N concurrent accesses.

Q9.

What is virtual memory and why is it used?

Ans: Virtual memory is a memory management technique that gives processes the illusion of having more memory than physically available. It uses disk space as extension of RAM. Benefits: Run programs larger than physical RAM, isolation between processes, easier memory allocation, demand paging (only load needed pages).

Q10.

What is paging? Explain the page table.

Ans: Paging divides physical memory into fixed-size **frames** and logical memory into same-size **pages**. The OS maintains a **page table** per process that maps page numbers to frame numbers. Address translation: Physical Address = (Frame number * Frame size) + Offset. Eliminates external fragmentation.

Q11.

What is thrashing?

Ans: Thrashing occurs when a process spends more time handling page faults (swapping pages in/out) than executing. Caused by: too many processes competing for few frames. The CPU utilisation drops drastically. Solution: reduce degree of multiprogramming, use working-set model, or use page fault frequency (PFF) to adjust frames.

Q12.

What is the difference between internal and external fragmentation?

Ans: Internal Fragmentation: Memory allocated to a process is larger than needed — wasted space inside allocated block. Occurs in fixed-size partitioning and paging.

Ans: External Fragmentation: Enough total free memory exists but it is not contiguous. Occurs in dynamic partitioning. Solution: Compaction.

Q13.

What are the different CPU scheduling algorithms?

Ans: FCFS: Simple queue order — convoy effect.

Ans: SJF/SRTF: Optimal average waiting time but needs future burst time knowledge.

Ans: Round Robin: Time quantum — good response for interactive systems.

Ans: Priority: Higher priority first — may starve low-priority processes (fix: aging).

Ans: MLFQ: Multiple queues with different priorities and time quantums.

Q14.

What is the critical section problem?

Ans: Critical section is a code segment that accesses shared resources and must not be executed by more than one process simultaneously. Three requirements for a solution: **Mutual Exclusion** (only one in CS), **Progress** (selection can't be postponed indefinitely), **Bounded Waiting** (limit on waiting time). Solved using mutex, semaphores, monitors.

Q15.

Explain the Producer-Consumer Problem.

Ans: A classic synchronisation problem: Producer creates items and puts in bounded buffer; Consumer takes items from buffer. Problems: Producer must wait if buffer full; Consumer must wait if buffer empty; Race condition on buffer. Solved using two semaphores: **empty** (initially N) and **full** (initially 0), plus a mutex for mutual exclusion.

Q16.

What is a system call?

Ans: A system call is the programmatic way a user-space process requests a service from the OS kernel. Types: Process control (fork, exit, wait), File management (open, read, write), Device management (ioctl), Information (getpid, time), Communication (pipe, socket). System calls switch from user mode to kernel mode (mode switch/trap).

Q17.

What is the difference between user mode and kernel mode?

Ans: User Mode: Restricted mode — processes run here; cannot directly access hardware or execute privileged instructions. **Kernel Mode**: Privileged mode — OS runs here; has full hardware access. The CPU switches to kernel mode via system calls (trap/interrupt). This dual-mode prevents user processes from damaging the system.

Q18.

What is spooling?

Ans: Spooling (Simultaneous Peripheral Operations On-Line): Data is stored in a buffer (disk/memory) to be sent to a slow device (like printer). CPU writes to spool; device reads from spool independently. Allows CPU to be busy while device is slow. Example: Print spooler — multiple print jobs queued on disk.

Q19.

What is the difference between RAID levels?

Ans: RAID 0: Striping — no redundancy, max speed.

Ans: RAID 1: Mirroring — full redundancy, 50% space efficiency.

Ans: RAID 5: Striping + distributed parity — good balance of speed, redundancy, efficiency. Requires min 3 disks.

Ans: RAID 10: Mirror + stripe — high performance and redundancy. Requires 4+ disks.

Q20.

What is a TLB (Translation Lookaside Buffer)?

Ans: TLB is a small, fast hardware cache that stores recent page table entries (virtual → physical address mappings). On each memory access, TLB is checked first: **TLB hit** = fast (no page table lookup). **TLB miss** = slow (access page table, update TLB). Effective Access Time = (TLB hit ratio × TLB time) + (TLB miss ratio × memory access time).

Q21.

What is the Dining Philosophers Problem?

Ans: 5 philosophers sit at a round table with 5 forks. To eat, a philosopher needs 2 forks (left and right). Problem: All pick up left fork simultaneously — deadlock (circular wait). Solutions: Allow only 4 philosophers to sit simultaneously; Pick up both forks atomically; Asymmetric solution (odd/even numbering); Monitor-based solution.

Q22.

What is the difference between hard links and soft links?

Ans: Hard Link: Directory entry pointing to same inode as original file. Cannot span file systems. File not deleted until all hard links removed. **Soft Link (Symbolic Link)**: Pointer to the path of original file. Can span file systems. If original deleted, symlink becomes dangling. Like a shortcut in Windows.

Q23.

What is LRU page replacement?

Ans: LRU (Least Recently Used) replaces the page that has not been used for the longest period. It is based on temporal locality — recently used pages likely to be used again. Implementation: Use a stack or counter. LRU is near-optimal but expensive to implement exactly in hardware. Approximations: Clock algorithm, LRU-K.

Q24.

What is Belady's Anomaly?

Ans: Belady's Anomaly is the phenomenon where increasing the number of page frames results in more page faults with the FIFO page replacement algorithm. Discovered by Laszlo Belady. Most other algorithms (LRU, Optimal) are monotone — more frames = fewer or equal page faults. FIFO is not monotone.

Q25.

What is the difference between monolithic kernel and microkernel?

Ans: Monolithic Kernel: Entire OS runs in kernel space (device drivers, file system, networking). Fast but large; a bug can crash the whole system. Examples: Linux, Unix.

Ans: Microkernel: Only essential services in kernel; rest in user space. Slower (more IPC) but more stable and secure. Examples: Mach, QNX, MINIX.

Q26.

What are zombie and orphan processes?

Ans: Zombie Process: Process that has finished executing but still has an entry in the process table (waiting for parent to call wait()). It consumes no resources except PID and PCB.

Ans: Orphan Process: Process whose parent has terminated before it. In Unix, orphans are adopted by the init process (PID 1) which calls wait() for them.

Q27.

What is demand paging?

Ans: Demand paging loads pages into memory only when they are needed (on demand), not all at once at process start. When a page is accessed that is not in memory, a **page fault** occurs: OS pauses process, finds the page on disk, loads it into a free frame, updates page table, then resumes process. Reduces startup time and memory usage.

Q28.

What is the difference between FCFS and SSTF disk scheduling?

Ans: FCFS: Serves disk requests in arrival order. Simple but can cause long seeks (poor performance). **SSTF (Shortest Seek Time First)**: Serves the request closest to current head position. Better throughput but may starve distant requests (not fair).

Q29.

What is an inode?

Ans: An **inode** (index node) is a data structure in Unix/Linux file systems that stores metadata about a file: file size, permissions, ownership, timestamps, and pointers to data blocks on disk. The inode does NOT store the filename — that is stored in directory entries. Each file has a unique inode number.

Q30.

What is a kernel and what are its types?

Ans: The kernel is the core of the OS that manages system resources and communication between hardware and software. Types: **Monolithic** (Linux — all in one), **Microkernel** (Mach, QNX — minimal kernel), **Hybrid** (Windows, macOS — mix of both), **Exokernel** (research — minimal abstraction), **Nanokernel** (very minimal, for embedded).

Q31.

What is the difference between logical and physical address?

Ans: Logical (Virtual) Address: Address generated by CPU; what the process sees. Also called virtual address. **Physical Address**: Actual location in main memory. The **Memory Management Unit (MMU)** translates logical to physical using the page table or segment table at runtime.

Q32.

What is Aging in scheduling?

Ans: Aging is a technique to prevent starvation in priority-based scheduling. It gradually increases the priority of processes that have been waiting a long time. Example: Every 15 minutes, increase priority of waiting process by 1. Eventually even a low-priority process will get CPU time. Used in Multilevel Feedback Queue schedulers.

Q33.

What is the working set model?

Ans: The working set model (Denning) defines the **working set** of a process as the set of pages referenced in the last Delta time units (the working set window). It is used to prevent thrashing: if the sum of all working sets > total frames, suspend some processes. Ensure each process has enough frames for its working set before running.

Q34.

What is a race condition?

Ans: A race condition occurs when the outcome of a computation depends on the relative timing/ordering of two or more concurrent processes. The system's correctness depends on who 'wins the race'. Example: Two threads incrementing a shared counter — both read 5, both write 6 (result: 6 instead of 7). Fix: synchronisation (mutex, semaphore).

Q35.

What are the differences between multiprogramming, multitasking, and multiprocessing?

Ans: Multiprogramming: Multiple programs in memory; CPU switches when one waits for I/O — improves CPU utilisation.

Ans: Multitasking (Time-sharing): Multiple tasks share CPU with rapid switching — gives illusion of simultaneous execution.

Ans: Multiprocessing: Multiple CPUs executing processes simultaneously in parallel — true parallelism.

Q36.

What is copy-on-write (COW)?

Ans: Copy-on-Write is an optimisation used in fork(). When a process forks, instead of copying the parent's memory immediately, both parent and child share the same physical pages. When either modifies a page, only then is a copy made. This saves time and memory when the child calls exec() immediately (common pattern).

Q37.

What is the difference between a semaphore and a monitor?

Ans: Semaphore: Low-level synchronisation primitive. Programmer must correctly call wait()/signal() — error-prone. **Monitor**: High-level construct (language-level in Java). Only one process active in monitor at a time. Uses condition variables (wait, signal). Automatic mutual exclusion — harder to misuse. Java's synchronized keyword uses monitors.

Q38.

What is swapping?

Ans: Swapping is the process of moving an entire process from main memory to secondary storage (disk) to free up RAM, and bringing it back when needed. Allows more processes than physical memory can hold. The area on disk used for swapping is called swap space. Swapping is slower than paging (moves entire process vs individual pages).

Q39.

What are the goals of CPU scheduling?

Ans: Maximise: CPU utilisation, Throughput (processes completed per unit time).

Ans: Minimise: Turnaround time (total time from submission to completion), Waiting time (time in ready queue), Response time (time from request to first response).

Ans: Different algorithms optimise different metrics depending on the system type (batch, interactive, real-time).

Q40.

What is a page fault and how is it handled?

Ans: A page fault occurs when a process accesses a page not in main memory. Handling: (1) OS receives trap, checks if valid address. (2) If valid: find free frame in physical memory. (3) Schedule disk I/O to load the page from disk into the frame. (4) Update page table. (5) Restart the instruction that caused the fault. Invalid access → segmentation fault (program error).

14. ✏■ My Personal Notes Space

Use these pages to write your own notes, doubts, and key takeaways after watching the video. Writing by hand (or annotating) greatly improves retention!

■ After watching the OS video — My Takeaways

What surprised me the most:

Topics I need to revise more:

Concepts I understood well:

■ Doubts & Questions to Clarify

Doubt 1:

Doubt 2:

Doubt 3:

Doubt 4:

■ Interview Prep — Self Assessment

Topics I'm confident about:

Topics to practise more:

Questions I got wrong — to redo:

■ Formula & Quick Reference Sheet

Turnaround Time =

Waiting Time =

Effective Access Time (TLB) =

Throughput =

■ Resources & Links

Shivansh's YouTube: youtube.com/@shivanshvasu

Website: theshivanshvasu.com

LinkedIn: linkedin.com/in/theshivanshvasu

Other references:

■■■